

DRC-Aid: Design-Rule Correction via Agentic Framework utilizing Inference-Time Large Language Models

Anushka Mukherjee

Purdue University
West Lafayette, Indiana, USA

Kang He

Purdue University
West Lafayette, Indiana, USA

Kaushik Roy

Purdue University
West Lafayette, Indiana, USA

Abstract

Resolving Design Rule Violations (DRVs) in layouts entails an iterative loop of geometric edits and verification. We present **DRC-Aid**, a closed-loop agentic framework that automates local DRC repair by formulating it as verification-in-the-loop search. To constrain the combinatorial geometric repair space, a deterministic Rule Engine converts physical verification tool-reported violations into a bounded menu of geometric edits. An off-the-shelf Large Language Model (LLM) evaluates local geometric context to select edits from this menu, with budgeted depth-first search and backtracking. Immediate feedback from verification tools such as *Calibre nmDRC/nmLVS* enforces geometric compliance and guards against electrical-topology degradation, while a global Memory Bank prevents cyclic re-exploration. Evaluated on FreePDK45 layouts containing DRVs, DRC-Aid achieves DRC-clean, LVS-equivalent repairs in ~92.5% of cases with a ~98% total violation reduction, while residual cases yield partially repaired LVS-equivalent candidates. Under an identical search and verification infrastructure, LLM-based selection outperforms random (54.4%) and deterministic-heuristic (83.3%) policies, with the gap widening on cases with six or more violations.

CCS Concepts

• **Hardware** → **Physical design (EDA); Design rule checking.**

Keywords

Electronic Design Automation (EDA), Design Rule Checking (DRC), Large Language Models (LLMs), Agentic Framework

1 Introduction

Physical verification ensures that Integrated Circuit (IC) layouts satisfy fabrication constraints. A critical component of this stage is Design Rule Checking (DRC), which verifies that layout geometry complies with the constraints imposed by the fabrication process [13]. During layout development, engineers routinely encounter numerous Design Rule Violations (DRVs), requiring repeated cycles of geometric modification and re-verification to reach a DRC-clean state [9, 14]. A central challenge is context-dependent decision-making. Resolving one violation can displace neighboring polygons, introducing new violations that extend the design cycle.

Existing machine learning approaches primarily target DRC hotspot prediction and violation localization [7, 8, 15]; they identify where violations occur but offer limited guidance on correcting them. The automated repair approach of [6] relies on simulated-annealing optimization without adaptive reasoning. Large Language Models (LLMs) are a compelling alternative given their demonstrated trade-off reasoning [17] and structured EDA tasks [1, 18, 21]. However, directly applying LLMs to physical design remains

error-prone: they hallucinate components and violate strict physical constraints [4, 5]. Recent literature therefore argues that such systems must operate as agents that maintain state, plan action sequences, invoke deterministic tools, and adapt in a closed-loop manner [2].

Building on this paradigm, we present **DRC-Aid**, a closed-loop agentic framework that shifts the LLM’s role from an unconstrained edit generator to a highly constrained decision-maker. DRC-Aid formulates local layout repair as *verification-in-the-loop search*: a deterministic **Rule Engine** converts each Calibre-reported violation into a bounded menu of executable geometric edits, an off-the-shelf LLM within the **Reasoning Agent (RAT)** selects an edit from among them, and the **Layout Management Toolkit (LMAT)** executes each edit and validates it via *Calibre*. A **Mega Controller (MC)** orchestrates this loop through budgeted depth-first search (DFS) with backtracking, supported by a global memory of visited states.

DRC-Aid targets the repetitive local correction loop encountered during transistor-level layout development, where an engineer identifies a geometric violation, applies a small modification, reruns DRC, and revises the edit when neighboring violations appear. We investigate whether this bounded but frequently repeated class of local corrections can be automated through a constrained, tool-grounded agentic workflow.

The key contributions of this work are:

- **DRC-Aid**, an agentic framework that automates local geometric DRC repair by coupling off-the-shelf LLM reasoning with verification-guided search in a closed loop.
- A deterministic **Rule Engine** that discretizes the combinatorial geometric repair space into a bounded, executable action menu, enabling targeted LLM spatial reasoning.
- A **budgeted DFS** search with backtracking and a **global memory bank** that prunes suboptimal branches and prevents cyclic re-exploration.
- Strict (DRC-clean, LVS-equivalent) repair in ~92.5% of cases with ~98% total violation reduction on FreePDK45 [20] test cases. Under identical infrastructure, LLM action selection outperforms random (54.4%) and heuristic (83.3%) baselines.

2 Background and Related Works

2.1 DRC in Physical Design

Physical design transforms a circuit representation into a fabrication-ready layout through placement, routing, and physical verification stages [10]. Verification primarily includes DRC to ensure geometric compliance and Layout-vs-Schematic (LVS) checks to confirm electrical equivalence [13]. A DRV occurs when layout geometry fails to satisfy the rules specified by the fabrication process.

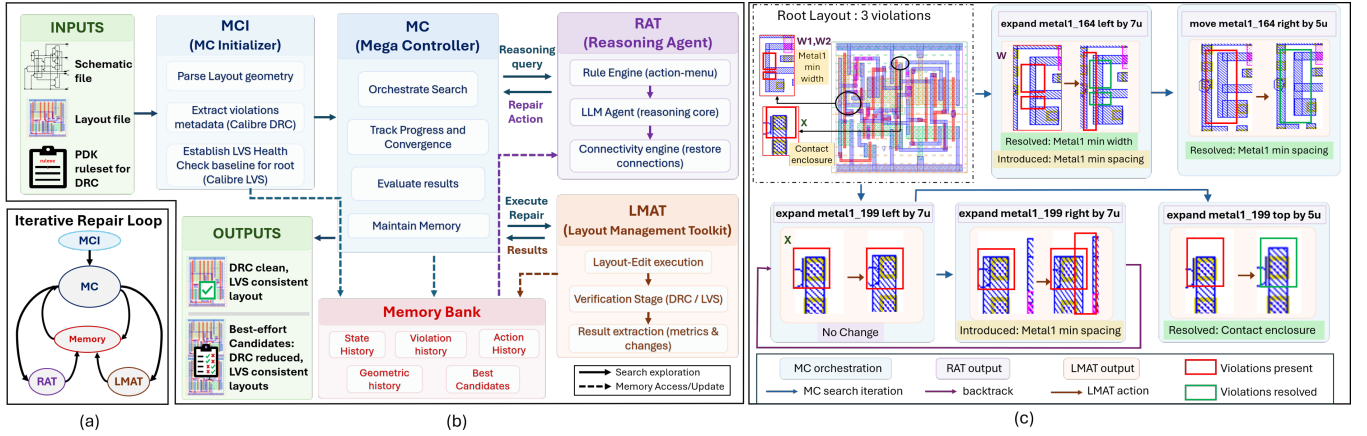


Figure 1: a) Iterative Search Exploration Loop of DRC-Aid b) Overview of the DRC-Aid Framework c) Illustrations of the iterative repair DRC-Aid conducts: the first row shows how fixing an initial violation caused another, which was then resolved; the second row shows that iterative steps induced a new violation instead of resolving the target; DRC-Aid backtracked and selected an alternative action that resolved it

Foundries define these rules within a Process Design Kit (PDK). Resolving DRVs is iterative: engineers use tools (e.g., *Calibre nmDRC*) to extract violation markers, determine necessary modifications, apply geometric edits, and re-verify to ensure no new violations are introduced. Although rule definitions and thresholds vary across technologies, commonly encountered categories include minimum width, spacing, enclosure, and overlap.

2.2 Existing Works

Existing research in DRC automation bifurcates into two categories: violation prediction and stage-specific repair. Prediction-focused works like [8] propose unsupervised methods to flag DRC hotspots pre-global routing. Similarly, CNN-based frameworks [7, 15] demonstrate that models can learn to predict violation density maps by extracting spatial features and congestion data. While these methods offer early identification of high-risk regions to guide global optimization, they are fundamentally diagnostic in nature: they identify *where* the violations may occur but do not provide solutions to fix them. Existing repair strategies are generally restricted to specific design stages. For instance, [11] embeds DRC-aware loops for routing closure, while [19] utilizes Reinforcement Learning to fix routing DRCs within standard cells. Although [6] uses an improved simulated annealing algorithm to repair violations in standard cell layouts, it treats repair as a cost-minimization problem using stochastic perturbations rather than context-aware spatial reasoning. Recent LLM-in-EDA works [1, 16, 17] have demonstrated the efficacy of LLMs in handling hardware tasks. However, they lack a closed-loop path from report to verified repair. DRC-Aid bridges this gap by introducing a context-aware, reasoning-driven iterative process by treating verification not as a downstream check but as the signal that drives each repair step.

3 Methodology

DRC-Aid is an iterative, context-aware framework designed to achieve autonomous DRC correction in transistor-level layouts.

Fig. 1(b) presents the core components and overall functionality of the framework. DRC-Aid utilizes **RAT (Reasoning Agent)** to navigate the repair search space; the **LMAT (Layout Management Toolkit)** interfaces with commercial EDA tools (*Cadence Virtuoso* and *Siemens Calibre*) for physical realization. The RAT and LMAT operate in tandem, with **MC (Mega-Controller)** guiding exploration. DRC-Aid operates in a 4-phase iterative loop: Observe, Reason, Execute, and Validate/Verify. In the initial **observation** phase, the MC’s initialization module (MCI) parses the layout geometry and performs a DRC check to identify violation data. This serialized context is injected into the RAT alongside a localized action-menu generated by the Rule Engine (refer to Sec. 3.2.1). The LLM then **reasons** to select the most promising fix. Following this, the Connectivity Engine runs a local connectivity-check on the current node against the original root layout (refer to Sec. 3.2.2) and, when needed, bundles a compensatory routing edit with the LLM-generated fix to maintain topological integrity. The **LMAT executes** the bundled fix on the layout and triggers a DRC **verification**. The resulting violation counts and modified geometry are returned to the MC, which updates the Memory Bank and proceeds to the **observation** phase of the next iteration. Fig. 1(c) illustrates how DRC-Aid resolves violations through an iterative exploration.

3.1 Mega-Controller (MC)

The Mega Controller orchestrates the layout repair search-space exploration by structuring the repair process as a Budgeted DFS strategy. MC guides the search towards a solution while preventing the RAT from stagnating in suboptimal states.

3.1.1 Budgeted DFS and anchor protection. To effectively explore the search space and prevent exhaustion of computational resources on unpromising branches of the search tree, MC enforces a depth-and-state-dependent budgeted exploration. Each node is assigned a budget inversely proportional to its depth from the root layout node, naturally decaying the exploration breadth as

depth increases. However, if a newly generated node constitutes an improved state (decreased violation count over the root), MC marks it as **anchor-protected**. This status grants an additional budget (proportional to its improvement) to encourage extensive exploration of that promising branch.

3.1.2 Backtracking. The MC utilizes backtracking to escape search stagnation. While the MC dictates the overall exploration strategy, it permits the RAT to explicitly request a backtrack to a previous node. Backtracking away from anchor-protected nodes is temporarily restricted to allow aggressive exploration around improving states. Additionally, MC initiates an independent backtrack to a previous node if a modified layout fails the mandated post-LMAT health-check (refer to Sec. 3.3.1), effectively pruning invalid candidates regardless of DRC status.

3.1.3 Graceful Termination. If a fully DRC-Clean and electrically equivalent state is unattainable, MC prevents complete failure by performing a global state evaluation across the search tree. During exploration, the memory bank (refer to Sec. 3.4) maintains a dynamic subset of **best-so-far** nodes that exhibit minimal violation count. Upon termination (budget exhaustion or a RAT stop request), MC performs topological health-checks (refer to Sec. 3.3.1) on these candidates. The framework then concludes by returning a list of **best-effort** solutions. This provides the physical design engineer with an improved starting point to resolve residual violations.

3.2 Reasoning Agent (RAT)

The RAT is the core decision-making element of the framework. It utilizes the reasoning ability of an LLM to navigate the layout repair search space. Instead of relying on unguided stochastic search, we incorporate a two-pass reasoning system. In the first pass, we inject dynamic state-aware context at runtime (action history, state memory), and a discrete menu of candidate actions generated by the Rule Engine (refer to Sec. 3.2.1). This allows the LLM agent to perform semantic evaluation and generate informed responses based on spatial and historical context. In the second pass, these semantically reasoned outputs are parsed to generate tool-calls to progress the search. Fig. 2(a) illustrates how the prompt evolves dynamically to capture this expanding state-aware context. The agent navigates the search space with five tools: MOVE, SHRINK, and EXPAND modify a shape’s position or dimensions along any of the four directions (top, bottom, left, right); BACKTRACK returns to a previous node when the current branch exhausts its budget or fails to improve; and STOP terminates exploration when residual violations resist extensive search or the budget is exhausted.

3.2.1 Rule Engine. If left unconstrained, the potential search space (S) for layout repair is combinatorially explosive for an LLM agent to navigate, requiring it to simultaneously select a candidate shape, edit type, direction, and magnitude:

$$S = C_s \times E_{tp} \times E_{dir} \times E_{mag},$$

where,

$C_s \subseteq C_{\text{layout}}$,	is the set of candidate shapes
$E_{tp} \subseteq \{\text{move, shrink, expand}\}$,	is the set of edit types
$E_{dir} \subseteq \{\text{right, left, top, bottom}\}$,	is the set of edit directions
$E_{mag} \subseteq \{k \cdot \Delta_{\text{grid}} \mid k \in \mathbb{Z}^+\}$,	is the set of edit magnitudes

To enable effective exploration, we introduce the Rule Engine, which discretizes the vast search space into a bounded set of valid geometric transformations, generating a library of potential fixes for each violation. A Calibre marker does not always uniquely identify the offending geometry; the engine therefore constructs rule-specific geometric hypotheses from shapes near the marker (facing pairs for spacing, deficient shapes for minimum width, inner-outer families for enclosure and containment, and intersecting pairs for overlap) and ranks them by marker proximity and geometric consistency. Algorithm 1 summarizes the overall flow: for each retained hypothesis, candidates are generated and classified as *legal*, *risk-annotated* (selectable, with flagged collateral risk), or *blocked*. Rule thresholds and layer relations are read from the ruledeck, and the generated actions span the local spacing, minimum-width, enclosure, extension, containment, and overlap checks evaluated in this work (the engine does not generate other topology-changing actions). While each individual action can resolve the targeted violation in isolation, it may inadvertently introduce new violations in the context of the layout. Consequently, the task of the LLM agent shifts from raw action-generation to strategic action-selection and trade-off evaluation.

3.2.2 Connectivity Engine. This engine acts as a post-processor to the agent’s raw output to ensure the proposed actions do not break essential connectivity established at the root node. When the agent chooses an action, this engine performs a local connectivity check. If connectivity is preserved, the raw output is passed on to the MC for later stages. However, if severed, the engine *bundles*

Algorithm 1 Rule Engine Candidate Generation

Require: Violations $\mathcal{V}$, layout geometry $\mathcal{G}$, ruledeck $\mathcal{R}$, action history $\mathcal{H}$
Ensure: Structured candidate menu $\mathcal{A}$

- 1: $\mathcal{A} \leftarrow \emptyset$
- 2: **for** each violation $v_i \in \mathcal{V}$ **do**
- 3: Read rule category, layers, threshold t_i from $\mathcal{R}$
- 4: Construct hypotheses $\mathcal{P}_i$ from nearby shapes on rule-relevant layers
- 5: Rank $\mathcal{P}_i$ by proximity and consistency; retain best matches
- 6: **for** each retained hypothesis $h \in \mathcal{P}_i$ **do**
- 7: Measure rule deficit $\delta(h)$ against t_i
- 8: **if** spacing **then** generate separation moves, facing-edge shrinks
- 9: **else if** width **then** generate deficient-edge expansions
- 10: **else if** enclosure **then** generate outer expansions, inner adjustments
- 11: **else if** overlap **then** generate separating moves or edge shrinks
- 12: **end if**
- 13: Snap magnitudes to manufacturing grid
- 14: Classify as legal / risk-annotated / blocked via editability, boundary, min-width, and action history ($\mathcal{H}$) checks
- 15: Add legal and risk-annotated candidates to $\mathcal{A}$
- 16: **end for**
- 17: **end for**
- 18: Merge equivalent candidates across markers; **return** $\mathcal{A}$

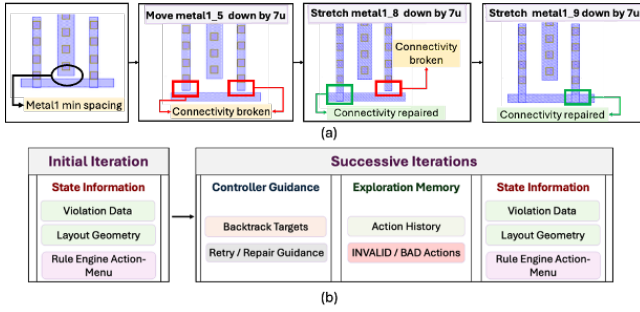


Figure 2: a) Connectivity Engine actions b) Dynamic Prompt Assembly from initial to successive iterations, incorporating exploration history and controller guidance

the agent’s output with a compensatory repair sequence to fix the broken connection. This *bundle* is passed on to the MC, which then enforces the LMAT operation. Compensatory bundles are not trusted: they pass through the same Calibre DRC/LVS verification as any other edit and are backtracked identically if they introduce violations or fail LVS. Fig. 2(a) illustrates how the connectivity repair is performed.

3.3 Layout Management Toolkit (LMAT)

The LMAT is the framework’s physical execution layer, translating RAT’s semantic decisions into physical layout modifications. To achieve real-time execution, we implement a Python-to-SKILL bridge that enables programmatic layout manipulations within *Cadence Virtuoso*. The MC transmits a parsed, SKILL-compliant instruction set, comprising discrete or bundled repairs, to the LMAT, which executes these edits to generate the new physical node by implementing the RAT’s proposed action.

3.3.1 Multi-stage verification. The LMAT performs verification checks to analyze the edit’s impact on the layout’s physical and topological integrity. To maintain industry-standard sign-off accuracy, LMAT uses *Siemens Calibre (nmDRC and nmLVS)* as the primary verification engine.

DRC evaluation: Every generated node undergoes a mandatory DRC evaluation (using *Calibre nmDRC*) to either verify whether all violations were resolved or extract the updated violation data required to drive subsequent iterations of the search.

LVS Health Checks: Although autonomous LVS repair is beyond the scope of this work, LVS is employed as an electrical-topology guardrail. A mandatory check (using *Calibre nmLVS*) is triggered for DRC-clean states, and for intermediate states where an edit introduces a new overlap between distinct polygons. If Calibre reports a mismatch relative to the root (device- or pin-count discrepancy, unintended short, or an open connection), the state is discarded and search continues for a better solution. This ensures that unintended topological violations are immediately caught.

3.3.2 Post-evaluation backtrack & MC feedback. The LMAT acts as the feedback mechanism for the MC’s search strategy. If any generated node fails the mandated LVS health check, it is

flagged invalid. This signals an immediate backtrack to the MC, which then prunes the non-functional branch, ensuring the search continues until a viable solution is found. A *strict success* therefore denotes a DRC-clean layout whose extracted topology remains LVS-equivalent to the original cell.

3.4 Global Memory

To refine the search in real-time, MC maintains a global memory bank that serves as a comprehensive index of all exploration artifacts. It tracks individual shape modifications to ensure layout-geometry consistency. Additionally, it maintains a state memory and node fingerprint database, logging visited states, executed actions and their respective efficacy. Actions already executed at a node, or flagged as *bad* or *invalid*, are recorded in a localized per-node memory. During the reasoning phase, this historical context is dynamically injected into the RAT’s prompt (Fig. 2(a)). Consequently, if the RAT attempts to propose such an action, it is forced to re-evaluate and select an alternative. This dynamic feedback loop effectively prevents oscillation between previously failed states.

4 Results

4.1 Experimental Setup

Evaluation benchmark: A public corpus of DRC-repair benchmark does not currently exist. Therefore, we evaluate DRC-Aid on a 60-layout test suite derived from DRC-clean FreePDK45 standard cells [20], constructed to span the rule families and compound-violation structure encountered in layout editing. Source polygons were decomposed into Manhattan rectangles, and eligible shapes and layers were perturbed by randomized amounts calculated relative to the ruledeck thresholds. To avoid a collection of independent single-error examples, additional violations were introduced in the neighborhoods of existing violations, producing interacting, compound cases. The resulting Calibre-reported markers define the test suite. DRC-Aid receives only the perturbed layout and Calibre-reported markers; it is given no knowledge of the injected operation, magnitude, seed, or original geometry, and its inputs match what an engineer sees mid-flow.

As summarized in Table 1, the test suite contains 285 initial markers spanning 26 checks across 8 layer families (n/p well, n/p implant, active, poly, contact, Metal1, Via1, Metal2) and is predominantly compound: 53 of 60 layouts contain multiple markers, 48 multiple rule checks, and 39 violations from multiple layer families. Repairs are thus performed in the presence of surrounding geometry and other violations, where a locally plausible edit may affect subsequent decisions. Only 5 of 60 layouts (8.3%) are cleared by a single root-state Rule-Engine action; the rest require multi-step correction and reasoning over interacting repairs, with 45–48 of 60 layouts requiring at least one backtrack across model–seed runs.

Models and execution: We deployed DRC-Aid on a single NVIDIA H200 GPU using two off-the-shelf LLMs, Qwen3-Next-80B-A3B-Instruct [22] and Gemma-4-26B-A4B-it [3] (hereafter Qwen-80B and Gemma-26B), served through vLLM [12] with 32k context (Qwen-80B used bitsandbytes quantization), top-*p* 0.95; temperature starts at 0.2 and is raised stepwise to at most 0.6 when the

Table 1: Benchmark characterization.

Scale	Value	Complexity	Value
Layouts	60	Multi-marker	53 (88.3%)
DRC markers	285	Multi-rule	48 (80.0%)
Rule checks	26	Cross-family	39 (65.0%)
Layer families	8	≥ 6 markers	19 (31.6%)
Markers/layout	Layouts (%)	Marker category	Markers (%)
1–2	30.0	Spacing (same/cross layer)	56.8 (37.2/19.6)
3–5	38.3	Enclosure	30.5
6–8	15.0	Minimum width	7.8
9–16	16.7	Overlap	4.9

Note: Layouts % over 60 layouts; markers % over 285 initial markers.

Table 2: Seed-wise framework stability. (All values in %)

Model	Metric	Seed A	Seed B	Seed C	Mean $\pm$ std
Gemma-26B	SSR	96.7	86.7	93.3	92.2 $\pm$ 5.1
	TVR	99.6	97.2	97.9	98.2 $\pm$ 1.3
	PVR	83.3	85.0	88.8	85.7 $\pm$ 2.8
Qwen-80B	SSR	91.7	93.3	93.3	92.8 $\pm$ 1.0
	TVR	98.6	97.8	98.9	98.4 $\pm$ 0.6
	PVR	80.0	74.1	78.6	77.6 $\pm$ 3.1

Note: SSR over the 60-layout suite (PSR = 100% – SSR); TVR over aggregate initial violations; PVR within Pareto cases.

agent fails to select a valid action. Per-node branch budgets decay with search depth under a global cap of 300 productive cycles. Neither model is fine-tuned. To capture model variation and sampling non-determinism, each model was evaluated on all 60 layouts across three independent seeds (360 model–layout–seed evaluations). All selection policies and ablations use the same layouts, search budget, and verification flow. As the implementation of [6] is not publicly available, we reimplement its improved simulated annealing (adaptive temperature decay, Metropolis acceptance, conflict-guided neighborhood proposal, and best-solution restoration) over our Rule Engine action-menu, with the violation count as objective and the same verification flow and budget as all other policies. Connectivity Engine is disabled to match the scope of [6].

4.2 Evaluation

We classify post-repair states into

$\mathcal{S}_{\text{strict}}$ (DRC-clean and LVS-equivalent to root, Sec. 3.3.1),
 $\mathcal{S}_{\text{pareto}}$ (DRC-reduced and LVS-equivalent), and
 $\mathcal{S}_{\text{drc}}$ (DRC-clean regardless of LVS, tracked for ablations).

Since every run achieved at least a partial LVS-safe reduction, $\mathcal{S}_{\text{strict}} \cup \mathcal{S}_{\text{pareto}} = \mathcal{T}$, the full test suite.

We report the **Strict Success Rate** $\text{SSR} = |\mathcal{S}_{\text{strict}}|/|\mathcal{T}|$,
Pareto Success Rate $\text{PSR} = |\mathcal{S}_{\text{pareto}}|/|\mathcal{T}|$, and for ablations, $\text{DCO} = |\mathcal{S}_{\text{drc}}|/|\mathcal{T}|$.

Violation clearance over a subset $\mathcal{X}$ is $\text{VR}(\mathcal{X}) = \sum_{i \in \mathcal{X}} (V_i^{\text{init}} - V_i^{\text{rem}}) / \sum_{i \in \mathcal{X}} V_i^{\text{init}}$, from which we report the **Total Violation Reduction** $\text{TVR} = \text{VR}(\mathcal{T})$ and the **Partial Violation Reduction** $\text{PVR} = \text{VR}(\mathcal{S}_{\text{pareto}})$.

As shown in Table 2, DRC-Aid reaches $\sim 92\%$ SSR across both models with $\sim 98\%$ TVR, versus 1.6–6.6% SSR for static prompting (Table 3). The $\sim 7.5\%$ best-effort cases still reduce violations by 78–86% (PVR) while preserving LVS-equivalence, returning a verified partial repair rather than a failure. All edits were made within the layout boundary, thus additional area cost was not incurred. Every layout in the suite was strictly resolved by at least two of the six model–seed runs, indicating that residual failures reflect sampling non-determinism and the finite per-node budget rather than intrinsic unsolvability. Failed runs are distributed across the difficulty spectrum, including on layouts that other seeds resolve cleanly. Conditional on strict success, repairs take a median 8–9 Calibre nmDRC and 2 nmLVS calls, 4.4 min (Gemma-26B) / 11.4 min (Qwen-80B) (Table 4), and 40–44k input / 2.5–3.4k output tokens per repair (input tokens include the injected layout-geometry context (serialized shape coordinates, violation metadata, and Rule Engine action-menus) that grows with search depth). As reflected by comparing LLM and non-LLM policies at matched verification workloads, Gemma-26B’s inference overhead is negligible relative to EDA-tool variance; Qwen-80B adds 25–30 s per LLM generation, accounting for roughly half its wall-clock time.

4.3 Ablation Studies

Component ablation: Table 3 quantifies each module’s impact along TVR, DCO, and SSR. Across all configurations, **2-pass reasoning** consistently outperforms direct 1-pass tool-call generation, but the static prompting baseline fails regardless ($\text{SSR} \leq 6.6\%$). Adding **DRC-feedback iteration** reveals raw geometric capability (Qwen: 85.8% TVR, 81.7% DCO), but blind edits degrade topology (18.3% SSR). **Memory and pruning** prevents cyclic oscillation and trades blind clearance for topological integrity (Qwen DCO drops to 71.6% while SSR rises to 23.3%), yet the unconstrained action space remains too vast. Coupling the LLM with the **Rule Engine** lifts TVR above 98% and DCO to 95–97.8%, but geometric heuristics alone cap SSR at $\sim 74\%$. Finally, **LVS-incorporated iteration** preserves TVR/DCO (as they are LVS-independent metrics) while ensuring topological integrity, surging SSR to $\sim 92\%$ for both models. Automated repair thus requires the structured coupling of constrained spatial reasoning with multi-domain (DRC and LVS) verification – no single component suffices.

Selection-policy and prior-work baselines: To isolate the contribution of LLM selection from the Rule Engine, we compare three non-LLM selectors and a reimplement of the simulated-annealing repair of [6] under identical candidate generation, verification flow, search, and budget. The deterministic heuristic selects the top-ranked action under a fixed lexicographic priority over the same pruned menu: (i) violation-group coverage, (ii) primary over conflict-escape class, (iii) smaller edit magnitude, (iv) deterministic tie-break. The softmax policy samples over the identical heuristic ranking with $\tau=0.5$; $\tau \rightarrow 0$ recovers the deterministic ranker and $\tau \rightarrow \infty$ approaches uniform. This isolates whether the LLM’s advantage is merely sampling diversity over the same ranking.

Random selection solves 54.4%, so candidate generation alone does not trivialize the task; the deterministic heuristic reaches 83.3% at the lowest runtime. The LLM policies reach 92.2–92.8%, and the margin widens on ≥ 6 -marker layouts: 63.2% (heuristic) vs.

Table 3: Ablation Study of Framework Components Across Evaluation Cases (All values in %).

Configuration	Components Enabled						Gemma 26B			Qwen3 80B		
	2-Pass Reason	DRC F.B.	Iter. Search	Mem. Prune	Rule Eng.	LVS Check	TVR	SSR	DCO	TVR	SSR	DCO
LLM	–	✓	–	–	–	–	8.0	3.3	3.3	8.9	0	1.66
Prompting	✓	✓	–	–	–	–	21.0	6.6	6.6	10.6	1.66	1.66
DRC Iteration	–	✓	✓	–	–	–	17.7	5.0	11.6	21.2	6.7	21.7
	✓	✓	✓	–	–	–	62.7	30.0	46.6	85.8	18.3	81.7
Memory & Pruning	–	✓	✓	✓	–	–	19.8	20.0	31.6	23.0	11.7	35.0
	✓	✓	✓	✓	–	–	69.1	35.0	68.3	67.7	23.3	71.7
DRC-Aid (no LVS)	✓	✓	✓	✓	✓	–	98.2	73.8	97.8	98.4	74.5	95.0
DRC-Aid (Full)	✓	✓	✓	✓	✓	✓	98.2	92.2	97.8	98.4	92.8	95.0

Component Legend: 2-Pass: 2-Pass Reasoning (enabled-vs-disabled) DRC F.B.: DRC Feedback Iter. Search: Iterative Search Mem. Prune: Memory & Pruning Rule Eng.: Rule Engine LVS Check: LVS-incorporated iterations

Table 4: Action-selection policy ablation

Setting	Policy	SSR (%)			TVR (%)	DCO (%)	Time (mins)	DRC/LVS (calls)
		All	≥ 6 markers	< 6 markers				
Shared menu	Random	54.4 ± 8.2	33.3 ± 11.0	64.2 ± 7.0	89.5 ± 3.6	72.8 ± 3.5	5.0 [2.0, 11.7]	7 [3, 17.5] / 3 [2, 7]
	Deterministic	83.3	63.2	92.7	95.1	88.3	3.3 [2.0, 5.8]	5 [3, 8.3] / 1 [1, 2]
	Softmax ($\tau = 0.5$)	82.8 ± 2.5	63.2 ± 5.3	91.9 ± 1.4	91.5 ± 5.4	87.8 ± 1.9	3.4 [2.0, 7.3]	5.5 [3, 12.8] / 1 [1, 2]
Prior work	SA-RE	38.3 ± 6.0	24.6 ± 6.1	44.7 ± 6.1	86.0 ± 0.0	63.9 ± 2.5	4.3 [1.8, 10.5]	8 [2.3, 19.8] / 3 [2, 6]
DRC-Aid	Gemma-26B	92.2 ± 5.1	82.5 ± 8.0	96.7 ± 3.7	98.2 ± 1.3	97.8 ± 2.5	4.4 [1.8, 9.6]	8 [3, 17] / 2 [1, 5]
	Qwen-80B	92.8 ± 1.0	84.2 ± 5.3	96.7 ± 1.4	98.4 ± 0.6	95.0 ± 0.0	11.4 [4.0, 30.9]	9 [3, 19.8] / 2 [1, 5]

Note: SSR/TVR/DCO are mean ± std over three seeds, except for the seed-independent deterministic policy. Costs are median [Q1, Q3] over strict-success runs. All policies use the same Rule-Engine candidate menu, state budget, and Calibre verification tools. SA-RE omits connectivity-aware bundling to follow the scope of [6]; its comparison therefore evaluates the prior-work-style system rather than isolating action selection alone. Cost statistics are conditional on SSR runs; thus not unconditional efficiency comparisons.

82.5–84.2% (LLM). The softmax policy does not materially improve over the deterministic ranker, indicating that the observed LLM gain is not explained by simple stochastic sampling over this fixed ranking. The result is consistent with the LLM making use of state, geometry, and action-history context. The SA baseline behaves differently in kind. Because [6] optimizes DRC clearance without LVS-aware acceptance, DCO matches its scope; SA-RE reaches 63.9% DCO on our suite, compared with 95.0–97.6% for DRC-Aid, and 38.3% SSR once LVS-equivalence is required.

5 Conclusion

In this work, we presented **DRC-Aid**, a closed-loop agentic framework that automates local DRC repair by coupling a deterministic Rule Engine, budgeted DFS exploration with global memory, and LLM-based action selection with verification-in-the-loop. Evaluations demonstrate that DRC-Aid achieves ~98% total violation reduction, with ~92.5% of cases reaching complete resolution, a substantial improvement over static prompting, which peaks at 6.6% complete resolution. Under identical evaluation infrastructure, random selection yielded 54.4% completion, and a deterministic ranker yielded 83.3%, with the LLM’s advantage widening on multi-violation layouts.

The present evaluation verifies geometric legality and topology preservation on conventional geometric DRCs in FreePDK45. Natural next steps include extending the verification loop to parasitic- and timing-aware acceptance criteria, broadening rule-deck coverage across technologies, and enriching the action space with repairs for congested post-route scenarios. More broadly, DRC-Aid demonstrates that bounded, frequently repeated correction loops can be automated by constrained, tool-grounded agents.

Acknowledgments

This work was supported in part by the Center for the Co-Design of Cognitive Systems (CoCoSYS), a research center under the Joint University Microelectronics Program (JUMP) 2.0, a Semiconductor Research Corporation (SRC) initiative sponsored by DARPA, and by National Science Foundation.

References

- [1] Chen-Chia Chang, Chia-Tung Ho, Yaguang Li, Yiran Chen, and Haoxing Ren. 2025. DRC-Coder: Automated DRC Checker Code Generation Using LLM Autonomous Agent. In *Proceedings of the 2025 International Symposium on Physical Design* (Austin, TX, USA) (ISPD ’25). Association for Computing Machinery, New York, NY, USA, 143–151. doi:10.1145/3698364.3705347
- [2] Amur Ghose, Andrew B. Kahng, Sayak Kundu, and Bodhisatta Pramanik. 2026. Invited: Agentic AI for Physical Design R&D: Status and Prospects. In *Proceedings of the 2026 International Symposium on Physical Design* (Germany) (ISPD ’26).

- Association for Computing Machinery, New York, NY, USA, 133–141. doi:10.1145/3764386.3779612
- [3] Google AI for Developers. 2026. Gemma 4 model card. https://ai.google.dev/gemma/docs/core/model_card_4
 - [4] Khandakar Shakib Al Hasan, Syed Rifat Raiyan, Hasin Mahtab Alvee, and Wahid Sadik. 2026. CircuitLM: A Multi-Agent LLM-Aided Design Framework for Generating Circuit Schematics from Natural Language Prompts. arXiv:2601.04505 [cs.AI] <https://arxiv.org/abs/2601.04505>
 - [5] Zhuolun He and Bei Yu. 2024. Large Language Models for EDA: Future or Mirage?. In *Proceedings of the 2024 International Symposium on Physical Design* (Taipei, Taiwan) (ISPD '24). Association for Computing Machinery, New York, NY, USA, 65–66. doi:10.1145/3626184.3639700
 - [6] Wenli Huang, Bin Li, Wencho Liu, Zhaohui Wu, Zonghan Lei, Songting Huang, and Chaosheng Qin. 2025. A DRC Automatic Repair Strategy for Standard Cell Layout Based on Improved Simulated Annealing Algorithm. *Electronics* 14, 21 (2025). doi:10.3390/electronics14214267
 - [7] Wei-Tse Hung, Jun-Yang Huang, Yih-Chih Chou, Cheng-Hong Tsai, and Mango Chao. 2020. Transforming Global Routing Report into DRC Violation Map with Convolutional Neural Network. In *Proceedings of the 2020 International Symposium on Physical Design* (Taipei, Taiwan) (ISPD '20). Association for Computing Machinery, New York, NY, USA, 57–64. doi:10.1145/3372780.3375557
 - [8] Riadul Islam and Dhandeep Challagundla. 2025. Pre-Global Routing DRC Violation Prediction Using Unsupervised Learning. In *2025 23rd IEEE Interregional NEWCAS Conference (NEWCAS)*. 450–454. doi:10.1109/NewCAS64648.2025.11107147
 - [9] Sungyu Jeong, Chanhyong Lee, Minsu Kim, Iksu Jang, Myungguk Lee, Junung Choi, and Byungsub Kim. 2024. A Layout-to-Generator Conversion Framework With Graphical User Interface for Visual Programming of Analog Layout Generators. *IEEE Access* 12 (2024), 125942–125954. doi:10.1109/ACCESS.2024.3409738
 - [10] A.B. Kahng, J. Lienig, I.L. Markov, and J. Hu. 2011. *VLSI Physical Design: From Graph Partitioning to Timing Closure*. Springer Berlin. <https://books.google.com/books?id=DWUGHyFVpboC>
 - [11] Andrew B. Kahng, Lutong Wang, and Bangqi Xu. 2022. TritonRoute-WXL: The Open-Source Router With Integrated DRC Engine. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 4 (2022), 1076–1089. doi:10.1109/TCAD.2021.3079268
 - [12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) (SOSP '23). Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165
 - [13] L. Lavagno, G. Martin, I.L. Markov, and L.K. Scheffer (Eds.). 2016. *Electronic Design Automation for IC Implementation, Circuit Design, and Process Technology*. CRC Press. <https://doi.org/10.1201/9781315215112>
 - [14] Shuwei Li, Ckristian Duran, Ritara Takenaka, and Tetsuya Iizuka. 2025. Fully-Standard-Cell-Based Synthesizable Charge-Redistribution SAR ADCs and P&R-Based Layout Automation Framework. *IEEE Access* 13 (2025), 83450–83469. doi:10.1109/ACCESS.2025.3569273
 - [15] Jhen-Gang Lin, Yu-Guang Chen, Yun-Wei Yang, Wei-Tse Hung, Cheng-Hong Tsai, De-Shiun Fu, and Mango Chia-Tso Chao. 2023. DRC Violation Prediction with Pre-global-routing Features Through Convolutional Neural Network. In *Proceedings of the Great Lakes Symposium on VLSI 2023* (Knoxville, TN, USA) (GLSVLSI '23). Association for Computing Machinery, New York, NY, USA, 313–319. doi:10.1145/3583781.3590216
 - [16] Bingyang Liu, Haoyi Zhang, Xiaohan Gao, Zichen Kong, Xiyuan Tang, Yibo Lin, Runsheng Wang, and Ru Huang. 2025. LayoutCopilot: An LLM-Powered Multiagent Collaborative Framework for Interactive Analog Layout Design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 8 (2025), 3126–3139. doi:10.1109/TCAD.2025.3529805
 - [17] Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, Chris Cheng, Nathaniel Pinckney, Rongjian Liang, Jonah Alben, Himyanshu Anand, Sanmitra Banerjee, Ismet Bayraktaroglu, Bonita Bhaskaran, Bryan Catanzaro, Arjun Chaudhuri, Sharon Clay, Bill Dally, Laura Dang, Parikshit Deshpande, Siddhanth Dhodhi, Sameer Halepete, Eric Hill, Jiashang Hu, Sumit Jain, Ankit Jindal, Bruce Khailany, George Kokai, Kishor Kunal, Xiaowei Li, Charley Lind, Hao Liu, Stuart Oberman, Sujeet Omar, Ghasem Pasandi, Sreedhar Pratty, Jonathan Raiman, Ambar Sarkar, Zhengjiang Shao, Hanfei Sun, Pratik P Suthar, Varun Tej, Walker Turner, Kaizhe Xu, and Haoxing Ren. 2024. ChipNeMo: Domain-Adapted LLMs for Chip Design. arXiv:2311.00176 [cs.CL] <https://arxiv.org/abs/2311.00176>
 - [18] Shang Liu, Wenji Fang, Yao Lu, Jing Wang, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2025. RTLCode: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation Technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 4 (2025), 1448–1461. doi:10.1109/TCAD.2024.3483089
 - [19] Haoxing Ren and Matthew Fojtik. 2021. Invited- NVCell: Standard Cell Layout in Advanced Technology Nodes with Reinforcement Learning. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. 1291–1294. doi:10.1109/DAC18074.2021.9586188
 - [20] James E. Stine, Ivan Castellanos, Michael Wood, Jeff Henson, Fred Love, W. Rhett Davis, Paul D. Franzon, Michael Bucher, Sunil Basavarajiah, Julie Oh, and Ravi Jenkal. 2007. FreePDK: An Open-Source Variation-Aware Design Kit. In *2007 IEEE International Conference on Microelectronic Systems Education (MSE'07)*. 173–174. doi:10.1109/MSE.2007.44
 - [21] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2024. AutoChip: Automating HDL Generation Using LLM Feedback. arXiv:2311.04887 [cs.PL] <https://arxiv.org/abs/2311.04887>
 - [22] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>